QA report

Alexander Marggraf, Frederic Weyssow

May 11, 2025

1 Introduction

This report will address the quality assessment measures that our group (group 23) has implemented to ensure that the final game satisfies our demands. Our goal was to create a mostly bug-free game that can handle even the most bizarre of user inputs. To see whether we achieved this, we need to thoroughly test each method where possible, manually test the remaining areas from the outside, and use tools to gain more information about things like test coverage, best-practice violations, etc. To achieve this, we used various tools like JavaDoc, Log4j, JaCoCo, MetricsReloaded, JUnit, and SpotBugs. We will first go over our concept and concrete implementation, then we will talk about the results we have obtained, and finish with a discussion of these results.

2 Concept

Before starting to code, we decided on certain requirements our software should have. Our code should be as simple and as modular as possible to prevent bugs, allow for it to be easily changed, and make it reusable for other parts of the program. Any exceptions should be handled, and any unforeseen events should be anticipated and properly dealt with by the software. The code should be understandable to team members as well as outsiders. For debugging the software, a logger should be used to reveal information at runtime. This simplifies the process and also allows us to understand what is happening while the program is running. To further aid this progress, we wanted to test all methods to make sure all the functionality is present and that future changes do not destroy the functionality that is already there.

3 Execution

3.1 Tools

We used various tools to gain more information and to enforce certain rules. These include JavaDoc, Log4j, JUnit, MetricsReloaded, JaCoCo, and SpotBugs.

JavaDoc was used for every non-trivial method, trivial as in getters or setters, to properly describe what it does. This was done such that group members can easily understand each other's code.

Log4j is a logger and was mainly used to show the arrival, sending, and handling of messages, as the flow between client, and server was quite complex.

JUnit was our choice of software to conduct unit tests. We tried to test as many methods as possible and cover any scenario that one could think of. This allowed us to not only make sure our code was working correctly, but also catch oversights when updating the codebase.

MetricsReloaded is an IntelliJ plugin that can be used to obtain various metrics about a project. We focused on complexity, where we first looked at individual classes and then dove deeper to understand why certain classes showed high complexity. Furthermore, we tested how dependent each class is and how many dependencies each class has. Another metric we have obtained is how many lines of code each class has and how well we managed to cover the code with JavaDoc. To test the quality of our object-oriented design, we conducted a MOOD metrics analysis where we gained insight on how well data and methods are hidden and how interdependent classes are.

JaCoCo was used to gain insight into how much code was covered by unit tests. This was especially useful when actually writing the tests, as it was immediately apparent if a certain area of the code was not covered.

SpotBugs was used to see how much our code adhered to best-practice principles and to catch rather severe bugs in development.

All parts of the code that could not be tested using the aforementioned tools were tested using white- and black-box tests. This mainly includes the GUI, as it is hard to mock using testing software.

4 Results

In this section we will take metrics at different milestones and try to see how our software developed and how well we managed to adhere to the goals we set in the beginning. To achieve this, we will evaluate MetricsReloaded, JaCoCo, and SpotBugs at milestone 5 as well as check out earlier milestones and run MetricsReloaded there.

4.1 Milestone 5

We found 3 classes with high average operation complexity: Message, Main, and Server. Looking at the individual methods reveals that most of the complexity comes from big switch or if-else statements when creating and handling messages. Furthermore, the complex decisions that can be made during the login phase also lead to high complexity.

Looking at the dependency metrics, we found that all classes have reasonable cyclic dependencies, with the highest amount of cyclic dependencies being eight. This is important since high cyclic dependency might create code that is difficult to understand. Looking at dependencies, Main has the highest amount of dependencies, as it depends on eleven other classes, and the class GuiApp has the highest amount of dependents, meaning classes that depend on GuiApp.

Our AHF (attribute hiding factor) is quite high with 89.80%, meaning most data is hidden from outside access. The AIF (attribute inheritance factor) is low with only 1.54%, meaning that we have high redundancy in our code. The CF (coupling factor) is 26.32%, which is reasonable and shows us that we have good modularity within our code. The MHF (method hiding factor) is only 7.66%, which might suggest that our methods are too visible from the outside.

In total, the project encompasses 15'509 lines of code, where 6'685 are written java code. Additionally, we have written 1'219 lines of JavaDoc documentation.

Element	Missed Instructions	Cov.
game		19%
server		44%
message		77%
client		64%
Total	5'653 of 9'908	42%

Figure 1: Test coverage of our program

Having a look at the code coverage using JaCoCo it shows that only 42% of the code, excluding the GUI package, is covered with tests. Breaking this further down, we can see that the packages client and message have rather good coverage with 64% and 77% respectively, but the packages server and game are lacking with only 44% and 19% respectively. This low coverage is because we had difficulties testing the GUI parts, as we were unable to mock them. This is especially apparent since most classes in the package game are well tested, but Main only has a 8% coverage due to high dependency on the GUI.

Warning Type	Number
Bad practice Warnings	16
Experimental Warnings	4
Internationalization Warnings	4
Malicious code vulnerability Warnings	33
Multithreaded correctness Warnings	6
Performance Warnings	4
Dodgy code Warnings	9
Total	**76**

Figure 2: SpotBugs warning types

When analyzing the code using SpotBugs, the program finds a total of 76 warnings, with only 8 of them being high-priority, which is sufficient for our standard. Breaking it down further, we can see that 33 warnings are caused by getters that might expose internal data, but this is hardly avoidable. 16 of the warnings are bad-practice, where most of these are privacy warnings. The eight high-priority warnings consist of four default encoding warnings, where we are translating between String and Bytes, which relies on default encoding. This should ideally be fixed, but has not caused any problems regarding other platforms. The other four warnings reference accessing a static field

from an instance method, which should be avoided. This will be discussed further in the final part of this report.

4.2 Development over time

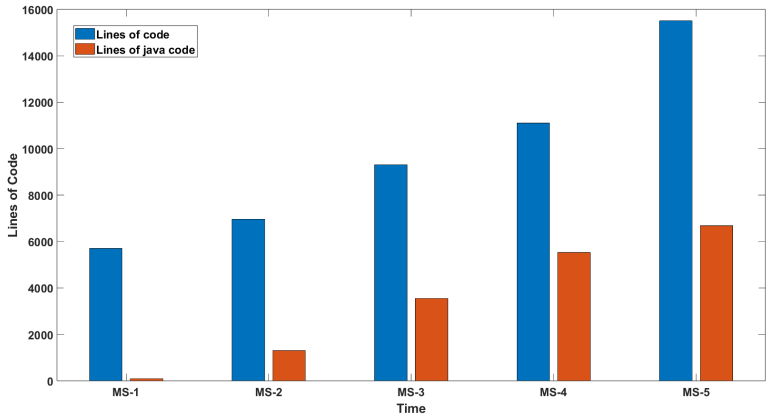


Figure 3: Lines of code over time

In Figure 3 we can see the development of the size of the project over time. Blue stands for overall lines of code, while orange stands for lines of java code. It is interesting to see that the growth from milestone to milestone is roughly the same for both total and java lines of code. Furthermore, it is apparent that the ratio of java to non-java files got bigger over time, ending at around 0.5.

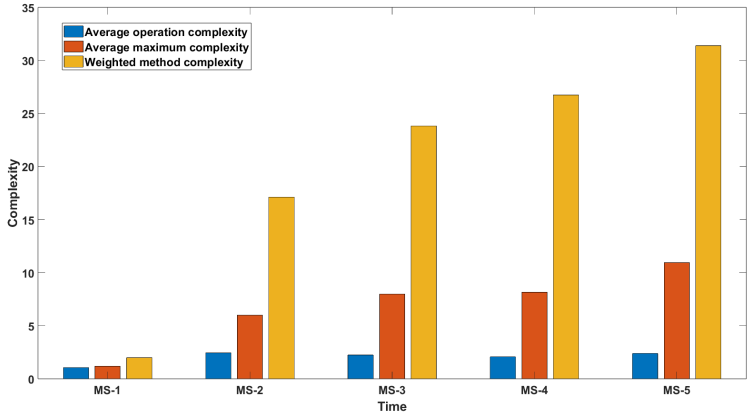


Figure 4: Complexity over time

In Figure 4 we can see the change of AOC (average operation complexity), AMC (average maxi-

mum complexity), and WMC (weighted method complexity). Regarding the AOC, it stays rather stable at around 2 to 3, which might indicate that our complexity did not change much during development, but the WMC shows another picture as it steadily increases. This discrepancy can be explained because most complexity is within only a few classes, even though many more classes influence the average. This then leads the AOC to not change much, even though the software is clearly becoming more complex. As for the AMC, it only rose slightly after milestone 2, which shows that most functionality was added to already existing methods and not implemented in new ones.

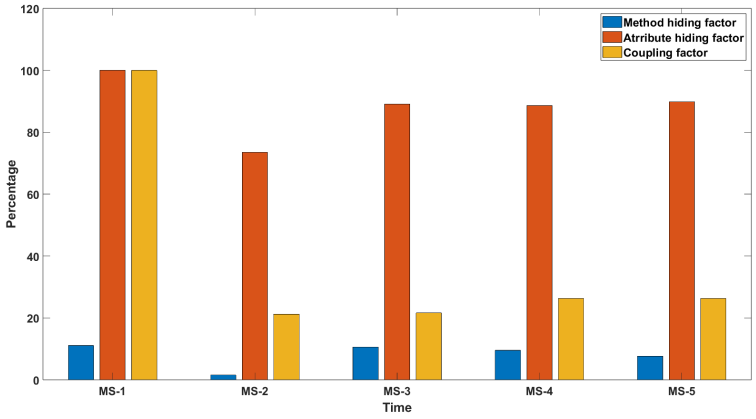


Figure 5: MOOD metrics over time

In Figure 5 we can see a stable high AHF, which means that we did a good job at hiding data inside classes and making it inaccessible from outside. The MHF is consistently low, which is because we mostly wrote public methods. The CF dips strongly from milestone 1 to milestone 2, but this is to be expected since only one class existed at milestone 1. However, it then slowly improves, which shows that we consistently did a good job at separating classes and keeping everything modular.

5 Discussion

Let us now discuss our results, see whether we met our goals or not, and talk about what we might do differently in the future.

Our first goal was to keep the code simple and modular to prevent bugs and allow for it to be easily changed and reused in the future. We achieved this quite well, as both the complexity and MOOD metrics analysis of MetricsReloaded showed good results. Most classes had good AOC and WMC and the ones that did not, were complex because of big switch statements. As this was by design, we achieved our goal here. As for the MOOD metrics analysis, we had a high AHF with $\sim$ 88% and a reasonable CF with $\sim$ 26%. This means that our data was well hidden from the outside and our classes have low interdependence, more exactly the CF states that on average only

every 4th class is used by another class. As for the AIF and MHF we got poor values that suggest that our code is highly redundant and that our methods are too visible from the outside. The latter is explainable since we mostly used public classes. This should be adjusted in a future project to better modularize classes and hide dangerous operations.

Our second goal was to handle any exceptions or unforeseen events. We managed to achieve this mostly, as we had only eight high-priority warnings using SpotBugs. However, we should have dealt with these, as all of them could be fatal to our software. One issue was using default encoding when converting from String to Bytes, which did not cause issues in our case, and the second issue was writing to static fields from an instance method. This should definitely have been avoided and the static keyword was overused in general throughout the project. In a future project, we should aim to only use static methods or fields when necessary.

Our third goal was to make the code understandable. We achieved this well, as we had over 1200 lines of JavaDoc documentation to explain any non-trivial methods. Additionally, our dependency metrics showed good results, implying that the code is easy to understand.

Our fourth goal was to use a logger to gain information at runtime. We did this sparsely, but we did use it for the entire flow of information between client and server. This did help tremendously when debugging, as this was a common source of mistakes, and we should definitely expand on the use of a logger in future projects.

Our final goal was to test all methods to make sure that correctness is preserved. As already mentioned, we did not cover enough of the code to satisfy our goal, as the GUI components were difficult to mock using JUnit. However, we did not test getter and setter methods, which furthermore makes a significant dent in our coverage. However, the methods we did test were tested quite thoroughly and we made sure to catch any mistakes. The methods we were unable to test using unit tests were tested using white- and black-box tests. This was sufficient, and we are confident that we caught most bugs. Having said this, we should not only vastly expand on the use of unit tests in future projects but also start using them much earlier in the process. Finally, our project lacks integration tests, which we mainly substituted with more white- and black-box tests.